Lektion 4: Wiederholung mit der for-Schleife – Musterlösung

Do Now

Was gibt dieses Programm aus? Kreuzen Sie an und begründen Sie Ihre Wahl Ihrer Partnerin oder Ihrem Partner gegenüber.

```
druck = 8
tuer = "zu"
if tuer == "zu" and druck < 6:
    print("Freigabe")
elif tuer == "zu" or druck < 6:
    print("Prüfen")
else:
    print("Gesperrt")
```

- ☐ A) `Freigabe`
- ☐ B) `Prüfen`
- ☐ C) `Gesperrt`
- ☐ D) `Freigabe und Prüfen`

B. Bei `and` müssen beide Bedingungen wahr sein (`8 < 6` ist falsch), bei `or` genügt eine. A: `and` wie `or` gelesen; C: `or` wie `and` gelesen; D: alle wahren Zweige laufen.

Predict (Vorhersagen)

Ein Messfahrzeug fährt gleichmäßig 15 m pro Sekunde. Das folgende Programm gibt aus, welchen Weg es nach jeder Sekunde zurückgelegt hat. Lesen Sie es, ohne es auszuführen.

```
geschwindigkeit = 15
for sekunde in range(1, 6):
    weg = geschwindigkeit * sekunde
    print(f"{sekunde} s: {weg} m")
print("Messung beendet")
```

Verfolgen Sie das Programm Schritt für Schritt in der **Variablentabelle**: eine Zeile pro Durchlauf mit den Werten der Variablen und der Ausgabe. Die erste Zeile ist als Beispiel ausgefüllt.

Durchlauf	sekunde	weg	Ausgabe
1	1	15	`1 s: 15 m`
2	2	30	`2 s: 30 m`
3	3	45	`3 s: 45 m`
4	4	60	`4 s: 60 m`
5	5	75	`5 s: 75 m`
nach der Schleife			`Messung beendet`

Run (Ausführen)

Führen Sie das Programm im Notebook aus. Entspricht die Ausgabe Ihrer Vorhersage? Falls nicht: Was ist anders?

Häufig erwartet: eine Zeile „6 s: 90 m“. Wird in Investigate 2 aufgegriffen.

Investigate (Untersuchen)

Nutzen Sie im Notebook die Kopie des Starterprogramms zum Ausprobieren.

1. In der Zeile `for sekunde in range(1, 6):` stecken drei Teile. Welcher ist das Schlüsselwort, welcher die Laufvariable, welcher der Zahlenbereich?

   `for` Schlüsselwort, `sekunde` Laufvariable, `range(1, 6)` Zahlenbereich.

2. Warum erscheint keine Zeile für 6 s? Das Struktogramm unten zeigt dasselbe Programm. Dort steht „bis 5“, im Programm `range(1, 6)`. Wie passt das zusammen?



   `range(1, 6)` erzeugt 1 bis 5, der Stoppwert 6 ist nicht enthalten. Das Struktogramm nennt den letzten Wert, `range` den Wert danach.

3. Ersetzen Sie `range(1, 6)` durch `range(5)`. Was ändert sich, und was bleibt gleich?

   Die Zählung beginnt bei 0 (0 s: 0 m bis 4 s: 60 m). Es bleibt bei fünf Durchläufen.

4. Ersetzen Sie den Zahlenbereich durch `range(0, 31, 10)`. Welche Werte nimmt `sekunde` an? Was bedeutet die dritte Zahl?

   0, 10, 20, 30. Die dritte Zahl ist die Schrittweite.

5. Rücken Sie die letzte Zeile `print("Messung beendet")` ein. Wie oft erscheint sie jetzt, und warum?

   Fünfmal. Eingerückte Zeilen nach dem Doppelpunkt bilden den Schleifenkörper und laufen in jedem Durchlauf.

Modify (Verändern)

Kopieren Sie das Starterprogramm in die Zelle im Notebook und ändern Sie es dort Schritt für Schritt weiter. Jeder Schritt baut auf dem vorigen auf.

1. Die Zahl der Sekunden soll nicht fest sein. Lesen Sie sie zu Beginn mit `input()` ein. Bei der Eingabe 3 sollen genau die Zeilen für 1 s bis 3 s erscheinen.
2. Das Fahrzeug fährt jetzt nicht mehr gleichmäßig, sondern beschleunigt: In der 1. Sekunde fährt es mit 3 m/s, in jeder weiteren Sekunde ist es um 3 m/s schneller. Das Programm soll für jede Sekunde den Weg ausgeben, den das Fahrzeug in dieser einen Sekunde zurücklegt. Bei der Eingabe 3 erscheinen die Zeilen „1 s: 3 m“, „2 s: 6 m“ und „3 s: 9 m“.
3. Das Programm soll zusätzlich den bisher zurückgelegten Gesamtweg ausgeben. Legen Sie dazu vor der Schleife `gesamt = 0` an und erhöhen Sie `gesamt` in jedem Durchlauf um den Weg dieser Sekunde. Bei der Eingabe 3 erscheinen die Zeilen „1 s: 3 m, gesamt 3 m“, „2 s: 6 m, gesamt 9 m“ und „3 s: 9 m, gesamt 18 m“.

```
# Schritt 1: Anzahl der Sekunden einlesen
geschwindigkeit = 15
anzahl = int(input("Anzahl der Sekunden: "))
for sekunde in range(1, anzahl + 1):
    weg = geschwindigkeit * sekunde
```

```
    print(f"{sekunde} s: {weg} m")
print("Messung beendet")

# Schritt 2: Weg je Sekunde beim Beschleunigen
geschwindigkeit = 3
anzahl = int(input("Anzahl der Sekunden: "))
for sekunde in range(1, anzahl + 1):
    weg = geschwindigkeit * 1
    geschwindigkeit = geschwindigkeit + 3
    print(f"{sekunde} s: {weg} m")
print("Messung beendet")

# Schritt 3: Gesamtweg mit Summenvariable
geschwindigkeit = 3
gesamt = 0
anzahl = int(input("Anzahl der Sekunden: "))
for sekunde in range(1, anzahl + 1):
    weg = geschwindigkeit * 1
    geschwindigkeit = geschwindigkeit + 3
    gesamt = gesamt + weg
    print(f"{sekunde} s: {weg} m, gesamt {gesamt} m")
print("Messung beendet")
```

Zu 1: `range(1, anzahl + 1)`, weil der Stoppwert nicht enthalten ist. Zu 2: Die Formel Weg = Geschwindigkeit · Zeit bleibt erhalten, die Zeit ist je Durchlauf 1 s. Die Reihenfolge zählt: Steht die Erhöhung vor der Rechenzeile, beginnt die Ausgabe bei 6 m. Typischer Fehler: Bleibt `* sekunde` stehen, erscheinen 3, 12 und 27 m. Die Kurzform `geschwindigkeit += 3` und die Lösung mit der Laufvariablen, `weg = 3 * sekunde`, sind gleichwertig. Zu 3: `gesamt = 0` steht vor der Schleife, sonst begänne die Summe in jedem Durchlauf neu. Bei der Eingabe 5 endet die Tabelle mit „5 s: 15 m, gesamt 45 m“.

Make (Erstellen)

Fahrschulen rechnen mit Faustformeln (v in km/h, Wege in m): Reaktionsweg = v / 10 · 3, Bremsweg = (v / 10)², Anhalteweg = Reaktionsweg + Bremsweg. Schreiben Sie im Notebook ein Programm, das für die Geschwindigkeiten 30, 40, 50 und so weiter bis 130 km/h eine Tabelle ausgibt.

So könnte die Ausgabe beginnen:

```
Geschwindigkeit: Reaktionsweg + Bremsweg = Anhalteweg
30 km/h: 9.0 m + 9.0 m = 18.0 m
40 km/h: 12.0 m + 16.0 m = 28.0 m
...
```

```
print("Geschwindigkeit: Reaktionsweg + Bremsweg = Anhalteweg")
for v in range(30, 131, 10):    # 30 bis 130 km/h in 10er-Schritten, 131 ist der Stoppwert
    reaktionsweg = v / 10 * 3
    bremsweg = (v / 10) ** 2
    anhalteweg = reaktionsweg + bremsweg
    print(f"{v} km/h: {reaktionsweg} m + {bremsweg} m = {anhalteweg} m")
```

Tests: 50 km/h ergibt 40.0 m, 100 km/h 130.0 m, letzte Zeile 130 km/h 208.0 m. Der Stoppwert muss über 130 liegen, z. B. 131.

Testen und Kommentieren

Prüfen Sie Ihr Programm mit den Testfällen im Notebook, besonders, ob die erste und die letzte Zeile erscheinen. Schreiben Sie hinter den Schleifenkopf einen Kommentar, welche Geschwindigkeiten die Schleife durchläuft und warum der Stoppwert so gewählt ist.

Erweiterung

- Hinter jede Zeile mit einem Anhalteweg über 100 m soll „Achtung“ erscheinen.

– Geben Sie nach der Tabelle aus, bei wie vielen Geschwindigkeiten der Anhalteweg über 100 m liegt.

```
print("Geschwindigkeit: Reaktionsweg + Bremsweg = Anhalteweg")
anzahl = 0
for v in range(30, 131, 10):    # 30 bis 130 km/h in 10er-Schritten
    reaktionsweg = v / 10 * 3
    bremsweg = (v / 10) ** 2
    anhalteweg = reaktionsweg + bremsweg
    zeile = f"{v} km/h: {reaktionsweg} m + {bremsweg} m = {anhalteweg} m"
    if anhalteweg > 100:
        zeile = zeile + "  Achtung"
        anzahl = anzahl + 1
    print(zeile)
print(f"Anhalteweg über 100 m bei {anzahl} Geschwindigkeiten")
```

„Achtung“ ab 90 km/h (108.0 m), also bei 5 Geschwindigkeiten. Die Bedingung im Schleifenkörper wird in jedem Durchlauf neu geprüft; der Zähler `anzahl` steht wie eine Summenvariable vor der Schleife.

Zusatzaufgabe: Sichere Geschwindigkeit bei Nebel

Ermitteln Sie die höchste Geschwindigkeit, bei der man bei Nebel noch vor einem Hindernis in Sichtweite anhalten kann. Das Programm liest die Sichtweite in m ein und prüft die Geschwindigkeiten in 5-km/h-Schritten bis 200 km/h.

So könnte ein Programmlauf aussehen:

```
Sichtweite in m: 50
Höchste sichere Geschwindigkeit: 55 km/h
```

Tipp: Setzen Sie die Zeile `%%diagramm` über Ihr fertiges Programm. Das Diagramm zeigt den Anhalteweg für jede geprüfte Geschwindigkeit.

```
sicht = float(input("Sichtweite in m: "))
sicher = 0
for v in range(5, 201, 5):
    anhalteweg = v / 10 * 3 + (v / 10) ** 2
    if anhalteweg <= sicht:
        sicher = v
print(f"Höchste sichere Geschwindigkeit: {sicher} km/h")
```

`sicher` merkt sich den letzten passenden Wert. Weil der Anhalteweg mit v wächst, ist das die höchste sichere Geschwindigkeit. Tests: 50 m ergibt 55 km/h, 100 m ergibt 85 km/h.

Schlüsselbegriffe – zum Nachschlagen und Lernen

Begriff	Erklärung	Wie in Python umsetzen?
Zählschleife (for)	wiederholt den Schleifenkörper für jeden Wert einer Zahlenfolge; die Zahl der Durchläufe steht vorher fest	`for i in range(3):` `  print("Hallo")`

range	erzeugt die Zahlenfolge; der Stoppwert ist nie enthalten	`range(4)  # 0 1 2 3`
		`range(1, 6)  # 1 2 3 4 5`
		`range(0, 31, 10)  # 0 10 20 30`

Schrittweite	dritte Zahl in range, Abstand zwischen zwei Werten	`range(30, 131, 10)`
Laufvariable	nimmt in jedem Durchlauf den nächsten Wert der Folge an und kann im Schleifenkörper verwendet werden	`for sekunde in range(1, 6):` `  print(sekunde * 15)`

Schleifenkörper	die eingerückten Zeilen nach dem Doppelpunkt; sie laufen in jedem Durchlauf. Die erste nicht eingerückte Zeile läuft einmal danach	`for i in range(3):`
		`  print("innen")  # 3-mal`
		`print("danach")  # 1-mal`

Summenvariable	wird vor der Schleife auf 0 gesetzt und in jedem Durchlauf erhöht	`gesamt = 0`
		`for x in range(1, 6):`
		`  gesamt = gesamt + x`
		`# Kurzform: gesamt += x`

Variablentabelle	verfolgt ein Programm Schritt für Schritt: eine Zeile pro Durchlauf mit allen Variablen und der Ausgabe	

Zählschleife im Struktogramm	„von 1 bis 5“ nennt den letzten Wert, den die Laufvariable annimmt	`für x von 1 bis 5`
		`  → range(1, 6)`
		`für x von 0 bis 30,`
		`Schrittweite 10`
		`  → range(0, 31, 10)`

Wichtige Hinweise zu Python

- Von 1 bis n zählt man mit `range(1, n + 1)`.
- Die Einrückung entscheidet, was wiederholt wird.
- Eine Summenvariable muss vor der Schleife stehen. Steht `gesamt = 0` im Schleifenkörper, beginnt die Summe in jedem Durchlauf neu.